Testing Documentation — **VeriSocial**

CSC1097 - Testing Documentation Dublin City University

Project Title: VeriSocial **Date Finished**: 02/05/25

Student 1 Name: Javier Alejandro Flores Baez **ID Number**: 21748521
Student 2 Name: Joaquin Escober **ID Number**: 21746259

Supervisor: Graham Healy

Testing Methods Section 1

Ad-hoc Testing 1.1

Below we have tested the performance of the different bot detection algorithms. Alongside our implementation of Isolation Forest, we have also used the out-of-the-box models from Scikit-learn. This included **OneClass SVM** and **Local Outlier Factor**. We decided to test the different bot detection methods to get an idea of the general accuracy and compare the performance against our own implementation.

The different bot detection algorithms are fed vectors as their inputs for calculation. In this case we represent users in the form of user activity vectors. Activities include standard user interactions on the platform which are then logged and stored in the database. This includes posting, commenting and liking. We then convert each user's logged activity into user vectors. These vectors include the max count of their activity, the standard deviation of each activity and the number of burst counts (times when the user displayed a frequent number of activities in a small time frame). Below is a sample of the user activity vectors.

```
{"user_id": "u1",  "total_actions": 20,  "avg_interval": 30,  "std_interval": 6.0,  "burst_count": 2},
{"user_id": "u2",  "total_actions": 22,  "avg_interval": 32,  "std_interval": 5.0,  "burst_count": 2},
{"user_id": "u3",  "total_actions": 18,  "avg_interval": 28,  "std_interval": 6.0,  "burst_count": 4},
{"user_id": "u4",  "total_actions": 19,  "avg_interval": 29,  "std_interval": 5.0,  "burst_count": 3},
 # Bot Example Below
{"user_id": "u5", "total_actions": 300, "avg_interval": 2, "std_interval": 1.0,  "burst_count": 100},
…
```

Table.1.1 - Sample of mock user data used during ad-hoc testing.

To test this we pre-prepared a list of mock users that will represent the user base when a system scans for bots. We prepared a total of 40 user activity vectors to be fed into the models. 8 of the 40 have irregular (or bot-like) values. Below is a graphical representation of the accuracy of each method.

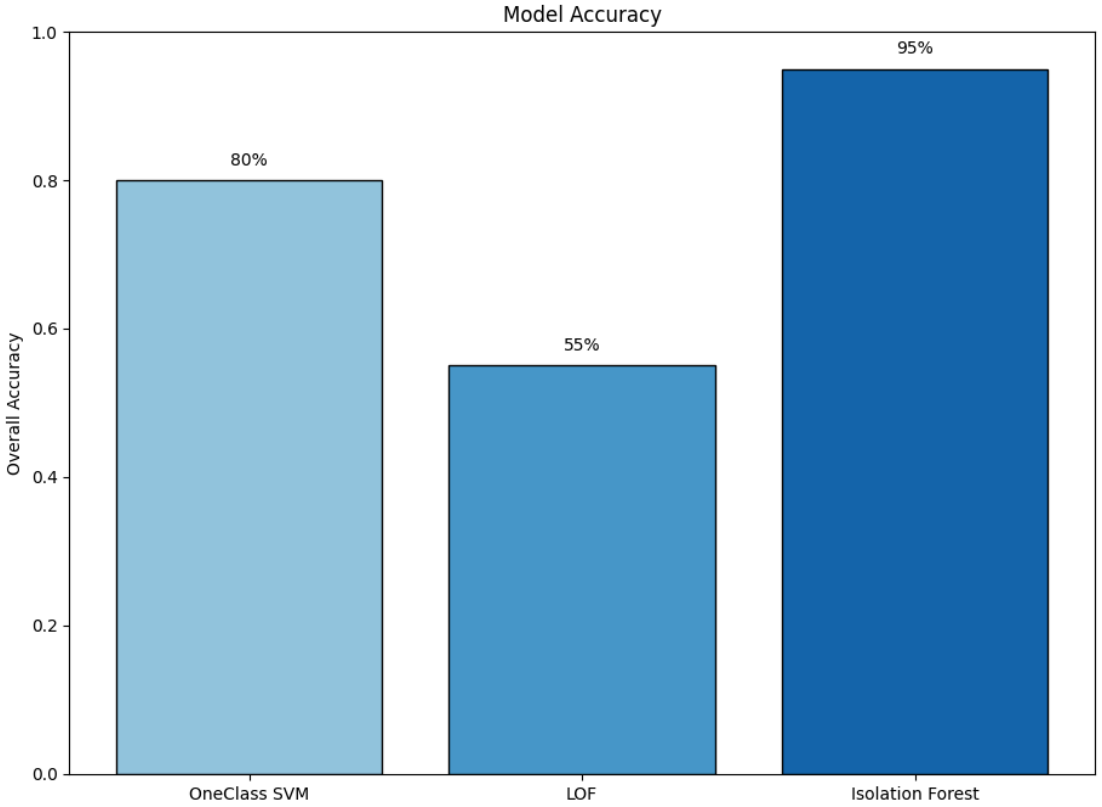


Fig.1.1 - Bot detection Algorithm accuracy (standalone).

Above we noticed that Isolation Forest works particularly well while Local Outlier Factor was performed averagely. Although accuracy is important for the detection of bots, it does not represent the core ideas / concepts of our VeriSocial project. For the purposes of this project, the current accuracy rates will be sufficient. To further this, we decided to implement a system that uses a mixture of these different methods. This was done to remove single model bias and the ability to configure weights of each bot detection method. As model accuracy is not our focus for this project, we implemented a configuration feature for the ability to change weighting of the different algorithms before combining the results. Below is a list of different weightings and their accuracies.

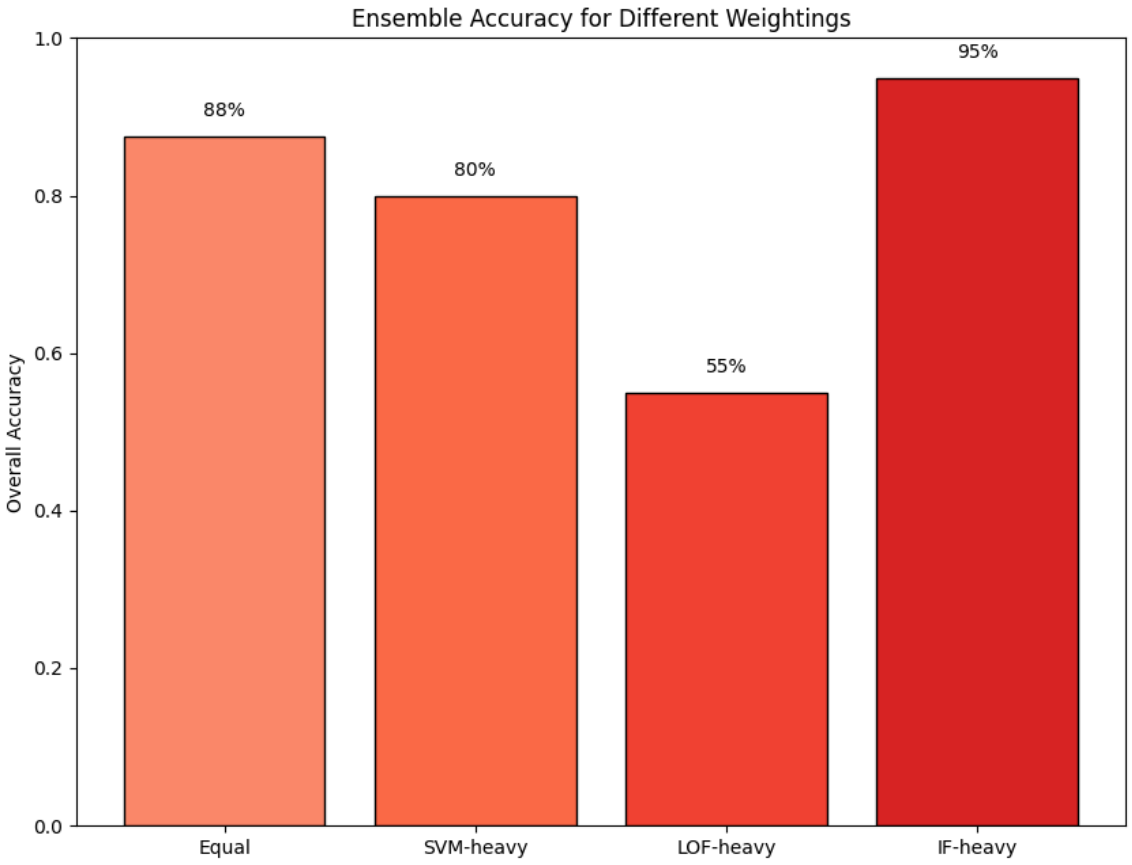


Fig.1.2 - Bot detection Algorithm accuracy (weighted and combined)

User Testing 1.2

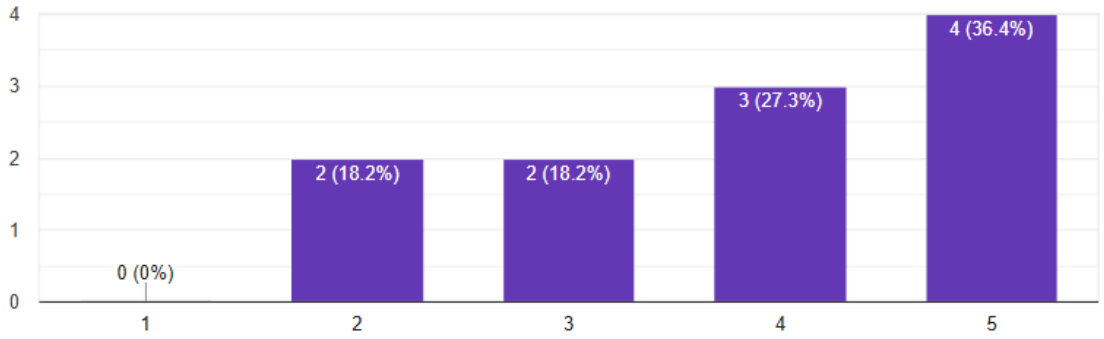
VeriSocial is designed with the intention of allowing online users to be able to engage and interact with our platform. To ensure that our web application meets user expectations and performs as intended, user testing was conducted. The primary goal of our user testing was to validate that the platform is suitable for use and also gain feedback from users based on their experience in using our platform.

For our user testing we aimed to get at least 10 participants to test the functionality of our application. According to the **Nielsen Norman Group** [LINK], testing with at least 5 participants would be enough to uncover most usability issues while being cost effective. In this case we were able to collect **11** responses. Our questions involved Statements and we measured how much each participant agreed on each statement based on their experience with our platform. Below are the results obtained from our user testing.

"I found the web application easy to understand and to navigate" How strongly do you agree with this Statement? 1-5

Copy chart

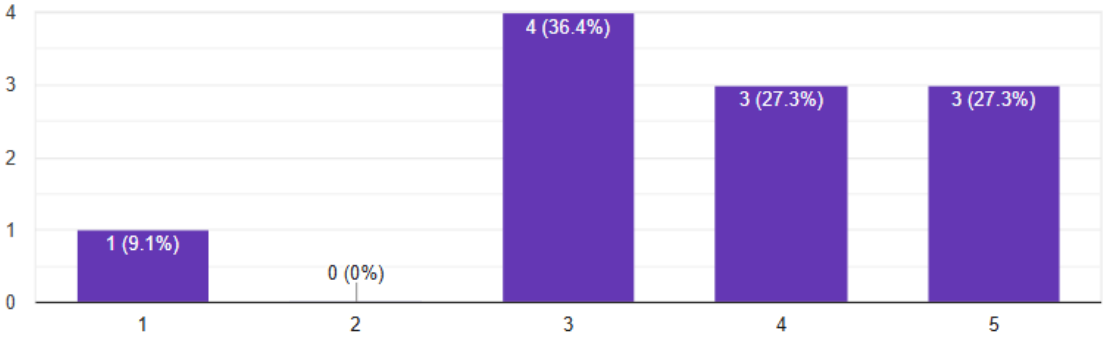
11 responses



"There's no AI generated content on VeriSocial compared to other social medias." How strongly do you agree with this Statement? 1-5

Copy chart

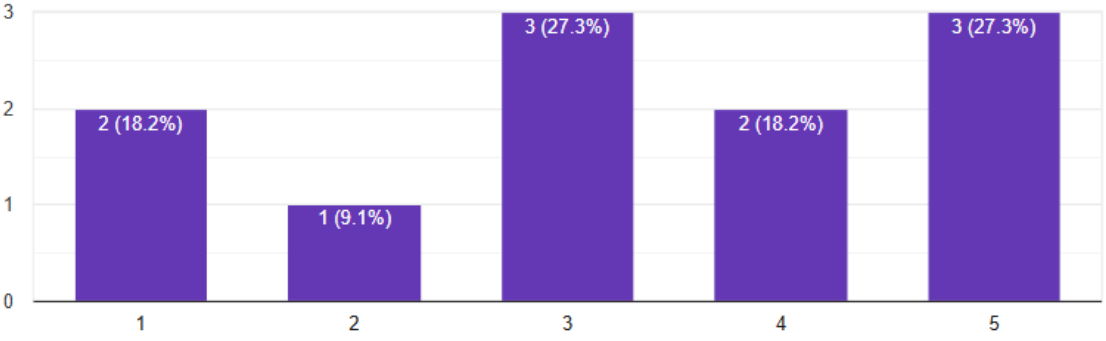
11 responses



"VeriSocial keeps the interactions between users authentic" How strongly do you agree with this Statement? 1-5

Copy chart

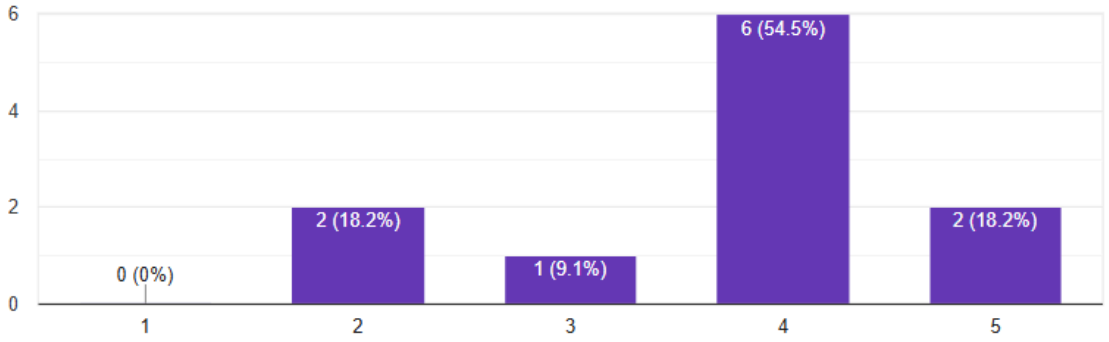
11 responses



"I feel that the application has given me enough tools for reporting and finding AI generated content." How strongly do you agree with this Statement? 1-5

Copy chart

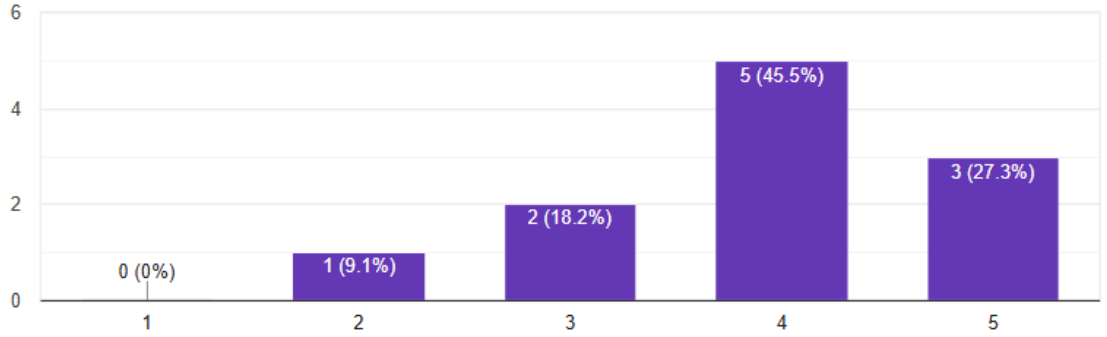
11 responses



"The bot score metric is most of the time accurate and its predictions align with my own opinion ." How strongly do you agree with this Statement? 1-5

Copy chart

11 responses



Based on the results above, we see that users are finding our key features easy to use and accessible. Our anti-AI features have also been received properly, with about 70% of the participants agreeing to find its estimated predictions and calculations to be accurate.

Unit Testing 1.3

(Frontend)

Frontend unit testing included testing component behaviours and mocking input data. **Vitest** along with the React testing library was used to create these unit tests. Example unit tests include loading fallbacks when the component is not fed any data. Rendering data and retrieving expected UI results. Calling redux functions successfully on certain actions or inputs. Below is the view of the unit tests when run. To run the tests simply run the command in the root directory of the Frontend client.

```
npm test
```

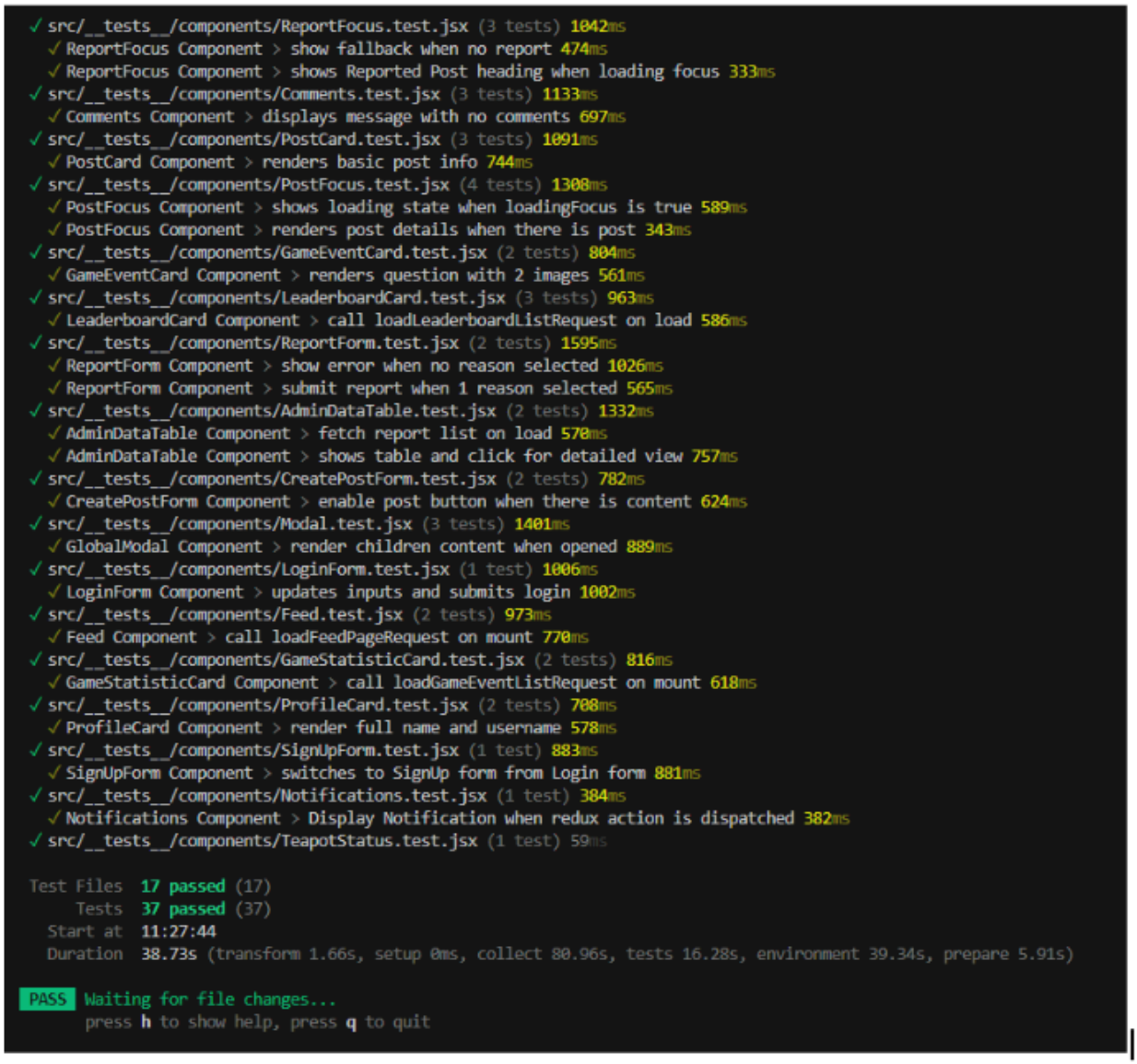


Fig.1.3 - Unit Test results on Frontend.